Generative AI Interview Questions & Answers

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to Generative AI

1 What is Generative AI ?

- Generative AI is a type of Artificial Intelligence that learns patterns from training data and can create new content such as text, images, audio, video, or code.
- It uses deep learning models (like Transformers) to understand the underlying structure and then generate similar but original outputs.

2 How does it work ?

- A generative model learns a probability distribution of the training data.
- During inference, it generates new outputs conditioned on a given prompt or context.
- It predicts the next token (in text) or next element (in images, audio, etc.) step by step.

3 Generative AI vs Traditional AI

Aspect	Generative AI	Traditional AI (Discriminative)
Goal	Create new content	Classify, predict or score inputs
Output	Text, image, audio, video, code, etc.	Label, category or numerical value
Model Type	Generative models (e.g., LLM, Diffusion)	Discriminative models (e.g., CNN, SVM)
Training	Learn data distribution	Learn decision boundary
Examples	ChatGPT, DALL·E, Stable Diffusion	Spam detection, Image classification

4 What is a Large Language Model (LLM) ?

- An LLM is a deep learning model trained on large amounts of text data to understand and generate human-like language.
- It is based on the Transformer architecture and predicts the next token in a sequence.
- Examples: GPT-4, Llama, Gemini, Claude, Mistral.

5 What is a Prompt ?

- A prompt is the input text or instruction given to a generative AI model.
- It can include a question, command, context, examples, or constraints.
- A well-designed prompt (prompt engineering) helps in getting better and more accurate responses.

6 Common Applications of Generative AI

- Chatbots and virtual assistants
- Text summarization and content generation
- Language translation
- Coding assistance (code generation, debugging)
- Image and video generation
- Document Q&A and information retrieval
- Creative writing, marketing content, and design

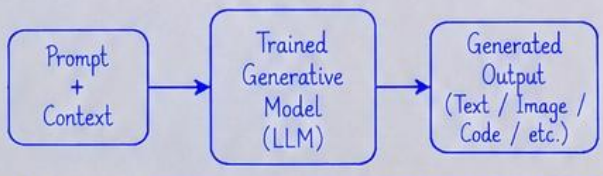
7 Benefits of Generative AI

- Helps in faster content creation and automation.
- Enables natural language interfaces for complex tasks.
- Improves productivity and reduces manual effort.
- Can assist in personalization and recommendation.
- Useful in education, research, healthcare, and business.

8 Limitations and Challenges

- May generate incorrect or hallucinated information.
- Can inherit bias from training data.
- Knowledge may be limited or outdated.
- Privacy and security risks (sensitive data leakage).
- High computational cost and latency for large models.
- Outputs should always be validated and not blindly trusted.

Generative AI Working Flow



Example

Prompt : Explain quantum computing in simple terms.

Model Output : Quantum computing uses qubits instead of classical bits. Unlike classical bits which are 0 or 1, qubits can be 0, 1 or both at the same time (superposition). This allows quantum computers to solve certain complex problems much faster than classical computers.

Interview Tip

- Explain that Generative AI models produce outputs based on learned patterns and probability, not fixed rules.
- Mention the importance of grounding, evaluation and human review, especially for critical use cases.

Key Takeaways

- Generative AI learns from data and creates new content.
- LLMs are based on Transformer architecture.
- Use case, benefits and limitations should be clearly understood.

Rahul Yadav

2. LLM Fundamentals

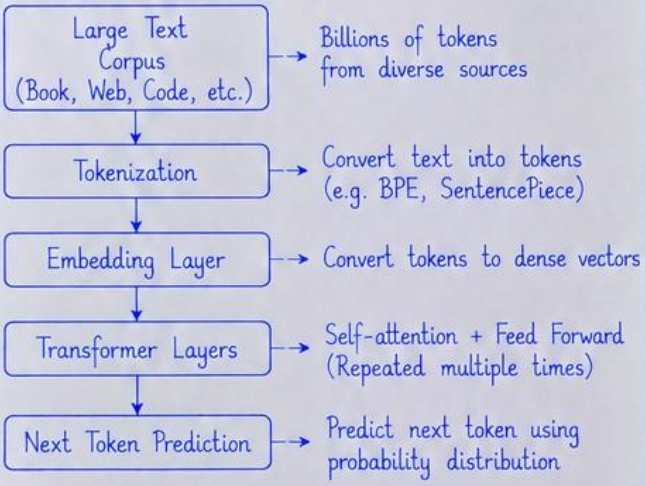
1 What is a Large Language Model (LLM)?

- A Large Language Model (LLM) is a type of Generative AI model trained on massive amounts of text data to understand, generate and manipulate human language.
- LLMs are based on the Transformer architecture and use self-attention mechanism to capture context and meaning.
- Examples: GPT-4, Llama, Gemini, Claude, Mistral.

2 How does an LLM work?

1. Input text is converted into tokens.
2. Tokens are converted into numerical embeddings.
3. Transformer layers process the tokens using self-attention.
4. The model predicts the next token based on context.
5. This process repeats to generate a complete response.

3 Training Process of an LLM



4 Tokenization

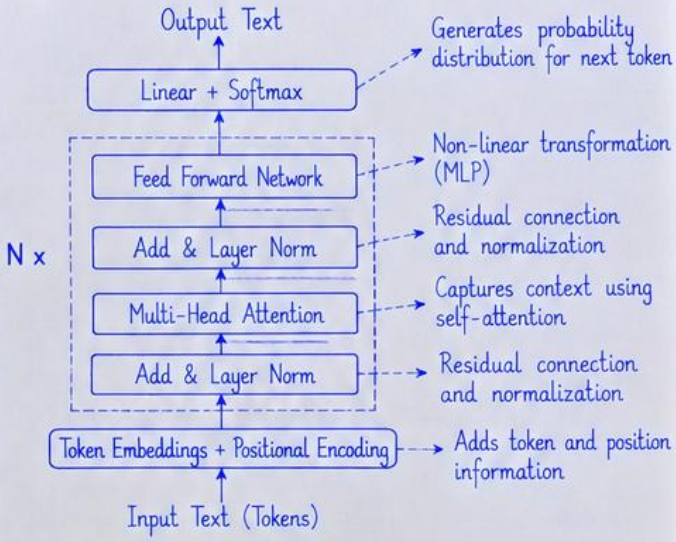
- Tokenization splits text into smaller units called tokens.
- Common tokenizers: Byte Pair Encoding (BPE), SentencePiece, WordPiece.
- Example:
  Text : "Generative AI is amazing!"
  Tokens : ["Generative", " AI", " is", " amazing", "!"]

5 Token, Embedding and Vocabulary

- Each token is converted into a high-dimensional vector called an embedding.
- Embedding captures the semantic meaning of the token.
- Vocabulary is the set of all possible tokens.
- Example:

Token	Token ID	Embedding (vector)
Generative	3204	[0.12, -0.34, ..., 0.56]
AI	417	[0.05, 0.23, ..., -0.11]
is	12	[-0.22, 0.11, ..., 0.09]

6 Transformer Architecture (High Level)



7 Key Concepts in LLMs

1. **Self-Attention** : Helps model understand context by attending to relevant tokens.
2. **Positional Encoding** : Adds position information to tokens.
3. **Multi-Head Attention** : Multiple attention heads to capture different relationships.
4. **Feed Forward Network** : Applies non-linear transformation.
5. **Layer Normalization** : Improves training stability.
6. **Residual Connection** : Helps in training deep models.

8 Pre-training vs Fine-tuning

Aspect	Pre-training	Fine-tuning
Objective	Next token prediction	Task-specific performance
Data	Large, general dataset	Smaller, labeled dataset
Cost	Very high	Relatively low
Use case	General language understanding	Domain-specific tasks (e.g. summarization, Q&A, code generation)

9 Popular LLM Models

Model	Organization	Key Features
GPT-4	OpenAI	High reasoning, multimodal
Llama 3	Meta	Open source, efficient
Gemini	Google	Multimodal, long context
Claude	Anthropic	Safe and helpful responses
Mistral	Mistral AI	Efficient, open source

10 Interview Tips

- Understand tokenization, embeddings and self-attention clearly.
- Be ready to explain how transformers work.
- Know the difference between pre-training and fine-tuning.
- Understand model limitations, biases and safety concerns.
- Discuss real-world applications and trade-offs.

Rahul Yadav

3. Transformer Architecture

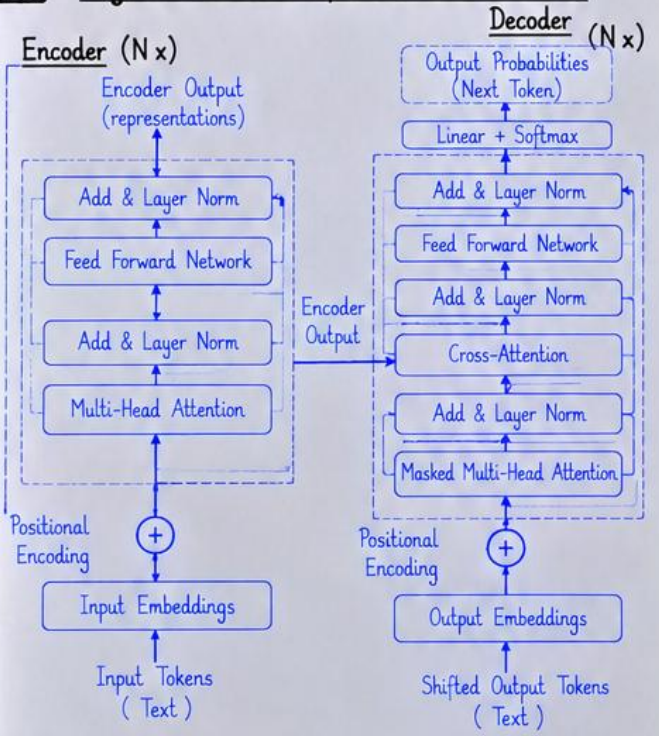
1 What is a Transformer ?

- Transformer is a deep learning model architecture introduced in the paper "Attention Is All You Need" (Vaswani et al., 2017).
- It is mainly used for natural language processing but also works for images, audio and other data.
- It uses self-attention mechanism instead of recurrent or convolutional layers.
- It enables parallel processing, better long-range dependency handling and higher performance.

2 Why was Transformer Introduced ?

- RNNs and LSTMs process data sequentially, which is slow and hard to parallelize.
- They struggle to capture long-range dependencies.
- Transformers use self-attention to directly look at all tokens in parallel.
- This results in better performance, scalability and easier training on large datasets.

3 High Level Transformer Architecture



4 Main Components

1. Input Embeddings : Convert tokens to vectors.
2. Positional Encoding : Adds position information.
3. Multi-Head Attention : Captures relationships between tokens.
4. Feed Forward Network (FFN) : Non-linear transformation.
5. Add & Layer Norm : Improves training stability.
6. Encoder Stack : Understands input sequence.
7. Decoder Stack : Generates output sequence (autoregressive).
8. Linear + Softmax : Converts output to probability distribution.

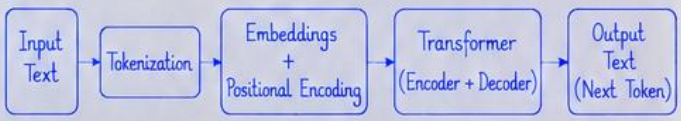
5 Encoder and Decoder Overview

- **Encoder** : Reads the entire input sequence and creates contextual representations for each token.
- **Decoder** : Generates output sequence token by token using masked self-attention and cross-attention with encoder output.

6 Transformer vs RNN/LSTM/CNN

Aspect	RNN / LSTM	CNN	Transformer
Processing	Sequential	Parallel (local)	Fully parallel
Long-range dependency	Difficult	Limited	Handles well (self-attention)
Training speed	Slow	Faster	Faster (parallel)
Model performance	Lower	Moderate	Higher
Architecture	Recurrent	Convolutional	Attention based

7 Input to Output Flow



8 Mathematical Formulation (High Level)

- Input sequence: $X = [x_1, x_2, \ldots, x_n]$
- Token embeddings: $E \in \mathbb{R}^{n \times d_{model}}$
- Add positional encoding: $Z = E + P$
- Pass through N encoder layers: $H = \text{Encoder}(Z)$
- Decoder generates output using masked attention and cross-attention: $Y = \text{Decoder}(H)$
- Final probability distribution: $P(y_t) = \text{Softmax}(W_o h_t)$

9 Example Input and Output

Input : "What is Generative AI ?"
Output : "Generative AI is a type of artificial intelligence that can create new content such as text, images, audio or code using deep learning models."

10 Key Takeaways

- Transformer uses self-attention instead of recurrence.
- It allows parallel processing and handles long-range dependencies.
- Encoder processes input, decoder generates output.
- Main components : embeddings, positional encoding, multi-head attention, feed forward network, layer normalization.
- It is the foundation for modern LLMs like GPT, BERT, Llama etc.
- Understanding this architecture is essential for AI interviews.

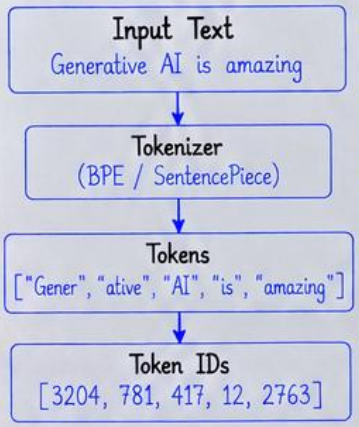
Rahul Yadav

4. Tokens, Embeddings and Context Window

1 What are Tokens ?

- Tokens are the smallest units of text that a language model can understand.
- A token can be a word, part of a word, or even a single character.
- LLMs use tokenization to convert text into tokens before processing.
- Example: "Generative AI is amazing"
  Tokens: ["Gener", "ative", "AI", "is", "amazing"]

2 Tokenization Process



- Uses Byte Pair Encoding (BPE) or SentencePiece.
- Splits text into subword tokens.
- Handles unknown words using subwords.
- Converts each token into a unique token ID.

3 Common Tokenization Algorithms

Algorithm	Used In	Key Idea
Byte Pair Encoding (BPE)	GPT, Llama	Merges frequently occurring pairs of bytes
SentencePiece	T5, Llama, Mistral	Language independent tokenization
WordPiece	BERT	Similar to BPE, uses likelihood
Unigram	Some LLMs	Selects best subword units from a vocabulary

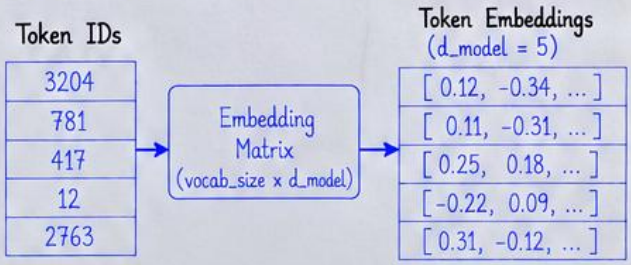
4 What are Embeddings ?

- Embeddings are dense numerical vectors that represent the meaning of tokens, sentences or documents.
- Tokens with similar meaning have similar embeddings in high-dimensional space.
- Embeddings capture semantic information and are learned during model training.
- Example:

Token	Token ID	Embedding Vector (dim = 5)
"Gener	3204	[0.12, -0.34, 0.56, 0.01, -0.23]
ative	781	[0.11, -0.31, 0.51, 0.03, -0.20]
AI	417	[0.25, 0.18, -0.62, 0.10, 0.44]
is	12	[-0.22, 0.09, 0.33, -0.11, 0.27]
amazing	2763	[0.31, -0.12, 0.48, 0.06, -0.35]

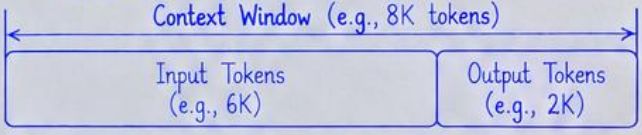
5 How Embeddings are Created ?

- Each token ID is mapped to a vector from an embedding matrix.
- The embedding matrix is a trainable parameter of the model.
- Embeddings are learned during pre-training using large text data.
- Similar tokens get similar vectors automatically.



6 What is Context Window ?

- Context window is the maximum number of tokens an LLM can consider at once.
- It includes both the input tokens and the output tokens.
- If the input exceeds the context window, older tokens may be truncated or compressed.
- Modern LLMs have larger context windows (e.g., 4K, 8K, 32K, 128K tokens).



7 Examples of Context Window in Popular Models

Model	Context Window	Notes
GPT-4	8K / 32K	Standard and extended
Claude 3	200K	Very large context window
Llama 3	8K / 70K	Extended context
Gemini 1.5	1M	Very large context (long documents)
Mistral	8K / 32K	Efficient and open source

8 Importance of Tokens, Embeddings and Context Window

1. Tokenization helps convert text into model-readable format.
2. Embeddings capture the meaning of text.
3. Context window determines how much information model can remember.
4. Larger context windows enable long document processing and better responses.
5. Better embeddings and tokenization improve model performance.

9 Key Takeaways

- Tokens are the basic units of text for LLMs.
- Embeddings are dense vectors that capture semantic meaning.
- Context window limits the number of tokens processed at once.
- Different models use different tokenization and context sizes.
- Understanding these concepts is important for prompts, RAG and long document applications.

Rahul Yadav

5. How Does RAG Work ?

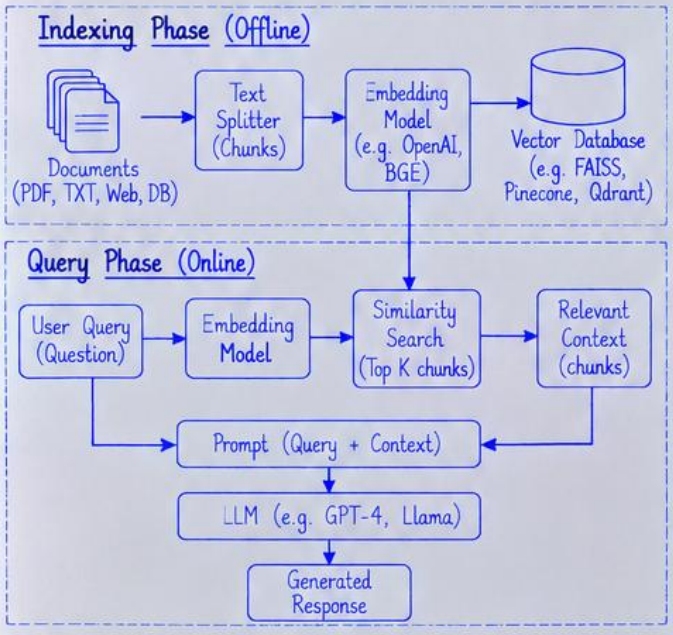
1 What is RAG ?

- RAG (Retrieval Augmented Generation) is a technique that combines information retrieval with large language models to generate more accurate, up-to-date and domain-specific responses.
- It retrieves relevant information from external knowledge sources (documents, databases, web, etc.) and provides it to the LLM as context.
- This helps reduce hallucinations and improves factual accuracy.

2 Why use RAG ?

- LLMs have limited knowledge and can generate incorrect or outdated information.
- RAG allows the model to use real, external data.
- Useful for domain-specific applications (company documents, manuals, research papers, etc.).
- Helps in building trusted and verifiable AI systems.

3 High Level Architecture of RAG



4 Step by Step Working of RAG

1. **Load Documents** : Collect data from PDFs, websites, databases, etc.
2. **Split into Chunks** : Divide documents into small chunks (e.g. 500 tokens) with optional overlap.
3. **Create Embeddings** : Convert each chunk into a vector using an embedding model.
4. **Store in Vector DB** : Save embeddings and metadata in a vector database (FAISS, Pinecone, Qdrant, etc.).
5. **User Query** : Convert the user query into an embedding.
6. **Retrieve Relevant Chunks** : Perform similarity search to find top-k relevant chunks.
7. **Create Prompt** : Combine the user query with retrieved context and pass to LLM.
8. **Generate Response** : LLM generates a grounded and factual answer using the context.

5 Example : RAG with Python (LangChain)

```python
# 1. Load and split documents
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("document.pdf")
documents = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=50)
chunks = splitter.split_documents(documents)

# 2. Create embeddings and store in vector DB
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
embeddings = OpenAIEmbeddings()
vector_db = FAISS.from_documents(chunks, embeddings)

# 3. Retrieve relevant documents
query = "What is Generative AI ?"
retriever = vector_db.as_retriever(search_kwargs={"k": 3})
retrieved_docs = retriever.get_relevant_documents(query)

# 4. Generate response using LLM with context
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA
llm = ChatOpenAI(model="gpt-4", temperature=0)
qa_chain = RetrievalQA.from_chain_type(llm=llm,
                                       retriever=retriever)
response = qa_chain.run(query)
print(response)
```

6 Types of RAG

Type	Description
Naive RAG	Basic retrieval of top-k chunks and pass to LLM.
Advanced RAG	Uses query rewriting, reranking and hybrid search.
Hybrid RAG	Combines keyword search (BM25) + vector search.
Agentic RAG	Uses AI agents to decide when and what to retrieve.
Graph RAG	Uses knowledge graphs for structured retrieval.
Multimodal RAG	Retrieves text, images, tables, etc.

7 RAG vs Fine-tuning

Aspect	RAG	Fine-tuning
Data Update	No retriaining needed	Requires retraining
Cost	Lower	Higher
Domain Knowledge	Uses external data	Learns during training
Factual Accuracy	Higher (grounded)	May hallucinate
Customization	Easy to update	Requires new training
Use Case	Dynamic / up-to-date info	Style / behavior change

8 Advantages and Disadvantages

Advantages	Disadvantages
• More accurate and factual responses • Can use latest and domain-specific data • Reduces hallucinations • No need to retrain the model • Cost-effective and flexible • Works with large unstructured data	• Requires a retrieval system • Performance depends on retrieval quality • Latency can be higher • Complex to design and optimize • Needs chunking and embedding pipeline

9 Key Takeaways

- RAG combines retrieval with generation to improve accuracy.
- It is widely used in real-world applications.
- Choice of chunk size, embedding model and vector DB is important.
- Different RAG variants exist for advanced use cases.
- RAG is a powerful technique for building reliable GenAI systems.

Rahul Yadav

6. Fine-Tuning vs. Prompt Engineering vs. RAG

1 What are the Three Approaches ?

- **Fine-Tuning** : Further training a pre-trained model on a specific dataset to adapt it to a particular domain or task.
- **Prompt Engineering** : Designing and optimizing input prompts to get better responses from an existing pre-trained model without changing the model weights.
- **RAG (Retrieval Augmented Generation)** : Combines external knowledge retrieval with LLM to generate accurate and up-to-date responses using relevant documents.

2 Key Differences (Overview)

Aspect	Fine-Tuning	Prompt Engineering	RAG
Model Training	Yes	No	No
External Data	Not required (in training)	Not used	Yes (retrieval)
Update Process	Retrain model	Change prompt	Update documents
Cost	High	Low	Moderate
Use Case	Domain specific tasks	General tasks with good prompts	Tasks needing latest/accurate information
Flexibility	Low	High	High
Accuracy (on domain data)	High	Medium	High

3 When to Use Each Approach ?

Scenario	Best Approach
General Q&A, simple tasks	Prompt Engineering
Domain specific tasks (e.g. medical, legal, finance)	Fine-Tuning or RAG
Need latest or frequently updated information	RAG
Limited budget and quick testing	Prompt Engineering
High accuracy for specific style/format	Fine-Tuning
Combining internal company data	RAG

4 Real-world Examples

1. **Fine-Tuning** : Training a model on company-specific support tickets to create a customer support chatbot.
2. **Prompt Engineering** : Using well-crafted prompts and examples to generate better code, summaries or analysis with GPT-4.
3. **RAG** : Building a chatbot that answers questions from internal PDFs, product manuals, or a knowledge base.

5 How They Work ?

Fine-Tuning



- Model learns new patterns from training data.
- Updates model weights.

Prompt Engineering



- Uses natural language instructions.
- No model training required.

RAG (Retrieval Augmented Generation)



- Fetches relevant information from external sources.
- Combines retrieved context with the user query.

6 Pros and Cons

Approach	Advantages	Disadvantages
Fine-Tuning	• High accuracy for domain tasks • Learns specific style and format • Works offline	• Requires labeled data • Expensive and time consuming • Less flexible • Needs retraining for updates
Prompt Engineering	• No training required • Quick and low cost • Very flexible • Works with latest models	• Performance depends on prompt quality • May give inconsistent outputs • Limited for complex domain tasks
RAG	• Uses external and up-to-date data • More accurate and grounded responses • No model training required • Flexible and scalable	• Requires retrieval system • More complex to build • Latency can be higher • Depends on document quality and chunking • Needs maintenance for updated data

7 Key Takeaways

1. Fine-tuning customizes the model for specific tasks.
2. Prompt engineering improves outputs using better instructions.
3. RAG provides factual and up-to-date information using external data.
4. Choose the approach based on your use case, data availability, budget and accuracy needs.
5. In many real-world applications, a combination of these approaches works best.

Rahul Yadav

7. Types of Large Language Models (Open-Source vs. Closed-Source)

1 What are Large Language Models ?

- LLMs are deep learning models trained on huge amounts of text data to understand, generate and manipulate human language.
- They are based on the Transformer architecture and use self-attention to capture context.
- LLMs can perform a wide range of tasks such as text generation, summarization, translation, Q&A, code generation and more.

2 Types of LLMs (Overview)

1. **Open-Source LLMs** : Freely available models whose weights, code and sometimes training data are accessible to the public.
2. **Closed-Source LLMs** : Proprietary models developed by companies, accessible through APIs with restricted model weights.

3 Open-Source vs Closed-Source Comparison

Aspect	Open-Source LLMs	Closed-Source LLMs
Model Weights	Publicly available	Not available
Access	Can run locally	Access via API
Customization	Fully customizable	Limited customization (via fine-tuning APIs)
Cost	Free (compute cost only)	Paid (token based)
Performance	Good, improving fast	Generally higher
Data Privacy	Full control (on-premise)	Data goes to provider
Examples	Llama, Mistral, Falcon, Gemma, Qwen	GPT-4, Claude, Gemini, Copilot

4 Popular Open-Source LLMs

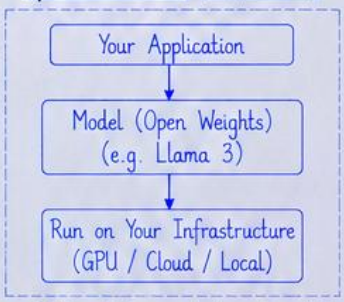
Model	Organization	Key Features
Llama 3	Meta	High performance, multiple sizes (8B, 70B)
Mistral	Mistral AI	Efficient, good reasoning
Gemma	Google	Lightweight, open weights
Falcon	TII	Good for research
Qwen	Alibaba	Strong multilingual support
Phi	Microsoft	Small and efficient

5 Popular Closed-Source LLMs

Model	Organization	Key Features
GPT-4 / GPT-4o	OpenAI	High reasoning, multimodal
Claude 3	Anthropic	Safe, long context window
Gemini 1.5	Google	Multimodal, large context (up to 1M tokens)
Copilot	Microsoft	Code generation, integrated with developer tools
PaLM 2	Google	Strong language understanding

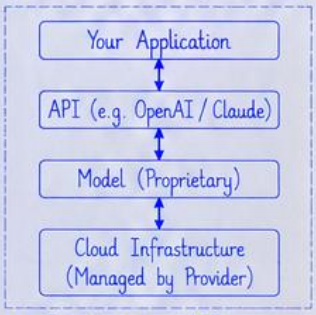
6 Architecture Differences

Open-Source LLM (Self-hosted)



- Full control over data and model
- Can fine-tune and customize
- Requires infrastructure and MLOps knowledge

Closed-Source LLM (API based)



- Easy to use via API
- No need to manage infrastructure
- More powerful and optimized
- Limited control and customization

7 Use Cases

Use Case	Open-Source (Best For)	Closed-Source (Best For)
Research / Learning	☑ Yes	Limited
Commercial Product	Yes (with infra)	☑ Yes
Sensitive / Private Data	☑ Best (on-premise)	Not suitable
High Accuracy Tasks	Good (with tuning)	☑ Better (GPT-4, Claude)
Cost Sensitive	☑ Better	Cost can be high
Quick Prototyping	Moderate (setup needed)	☑ Very easy

8 Strengths and Limitations

Aspect	Open-Source LLMs	Closed-Source LLMs
Strengths	• Free and flexible • Fully customizable • Can run locally • Transparent and auditable	• Higher performance • Optimized and well-tested • Easy to use (API) • Regular updates
Limitations	• Requires infrastructure • Need technical knowledge • May have lower performance • Responsible for maintenance	• Paid and can be expensive • No access to model weights • Limited customization • Data privacy concerns

9 Emerging and Special Types

1. **Multimodal LLMs** : Can understand and generate text, images, audio and video. (e.g. GPT-4o, Gemini)
2. **Small Language Models (SLMs)** : Lightweight models for specific tasks. (e.g. Phi, Gemma 2B)
3. **Domain Specific LLMs** : Trained or fine-tuned for specific domains like healthcare, finance, legal, etc.
4. **Mixture of Experts (MoE)** : Activates only relevant experts to improve efficiency. (e.g. Mixtral)
5. **Instruction Tuned LLMs** : Optimized to follow instructions better. (e.g. Llama 3 Instruct)

10 Key Takeaways

- Open-source LLMs are great for learning, research and custom use cases.
- Closed-source LLMs provide higher performance and are easy to use.
- Choose the model based on your requirements, budget, data privacy and infrastructure availability.
- Both types are useful and are being widely used in real-world applications.

Rahul Yadav

8. PEFT Methods (LoRA, QLoRA and Other Techniques)

1 What is PEFT ?

- PEFT (Parameter-Efficient Fine-Tuning) is a set of techniques to efficiently adapt a pre-trained model for a specific task.
- It trains only a small number of extra parameters instead of updating all model parameters.
- It reduces memory usage, training time and cost while maintaining good performance.
- PEFT is widely used for LLMs like Llama, Mistral, Phi, etc.

2 Why use PEFT ?

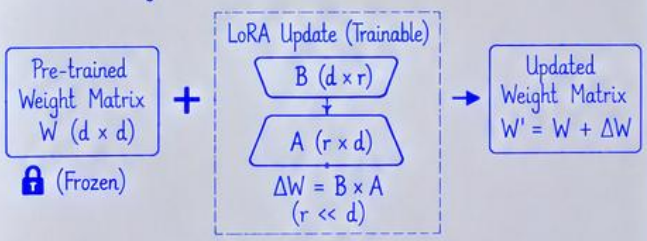
- Full fine-tuning is expensive and requires high GPU memory.
- PEFT methods train only a small portion of parameters.
- It makes it possible to fine-tune large models on consumer GPUs (e.g., 8GB, 16GB).
- Useful for domain adaptation, custom assistants and low-cost experimentation.

3 Types of PEFT Methods (Overview)

Method	Key Idea	Parameters Trained	Use Case
LoRA	Low-Rank Adaptation	Very few (1-10%)	General purpose fine-tuning
QLoRA	4-bit Quantization + LoRA	Very few (1-10%)	Large model fine-tuning on low-end GPUs
Prefix Tuning	Train prefix tokens	Very few	Text generation, classification
Prompt Tuning	Train soft prompts	Very few	Simple tasks
Adapter Tuning	Add small adapter layers	Few (adapter layers only)	NLP and vision tasks
IA^3	Learnable scaling vectors	Very few	Efficient fine-tuning
BitFit	Train only bias parameters	Very few	Lightweight fine-tuning

4 LoRA (Low-Rank Adaptation)

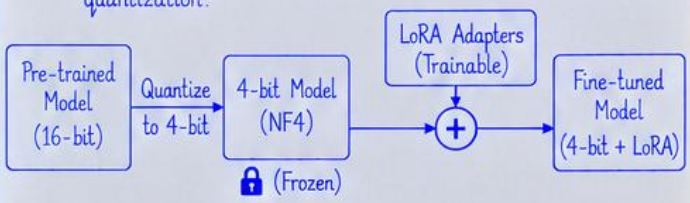
- LoRA adds low-rank matrices to the existing weight matrices and only trains those small matrices.
- Original weight matrix W is frozen.
- A low-rank update $\Delta W = B \times A$ is added.
- Final weight: $W' = W + \Delta W$



- r is the rank (e.g., 4, 8, 16).
- Only A and B are trained, which are much smaller than the original matrix.

5 QLoRA (Quantized LoRA)

- QLoRA combines 4-bit quantization with LoRA.
- The base model is loaded in 4-bit (using bitsandbytes).
- LoRA adapters are trained on top of the quantized model.
- It significantly reduces GPU memory usage (e.g., 7B model can be fine-tuned on a single 16GB GPU).
- It uses techniques like NF4 (NormalFloat4) and double quantization.



6 Other PEFT Techniques

Method	How it Works	Advantages
Prefix Tuning	Trains continuous prefix tokens prepended to input.	Simple and effective for generation tasks.
Prompt Tuning	Trains soft prompts instead of model weights.	Very lightweight and easy to use.
Adapter Tuning	Adds small adapter layers between existing layers.	Modular and reusable adapters.
IA^3	Learns small scaling vectors for attention and FFN layers.	Very efficient with good performance.
BitFit	Trains only bias parameters in the model.	Extremely simple and lightweight.

7 Comparison: Full Fine-tuning vs PEFT

Aspect	Full Fine-tuning	PEFT (LoRA / QLoRA etc.)
Parameters Trained	All parameters	Very few parameters
GPU Memory	High	Low
Training Time	Long	Short
Cost	High	Low
Performance	Best (for large data)	Comparable (for most tasks)
Flexibility	Less flexible	More flexible
Use Case	Large datasets, high resources	Domain adaptation, small datasets, low resources

8 Real-world Examples

Model	PEFT Method	Use Case
Llama 3 8B	QLoRA	Custom chatbot, domain-specific QA
Mistral 7B	LoRA	Instruction tuning, summarization
Phi 2	Adapter Tuning	Lightweight applications
BERT / T5	Prefix / Prompt Tuning	Classification, text generation
Stable Diffusion	LoRA	Image generation (text-to-image)

9 Key Takeaways

1. PEFT helps fine-tune large models efficiently.
2. LoRA is simple and widely used.
3. QLoRA enables fine-tuning of large models on low-end GPUs.
4. Different PEFT techniques are suitable for different use cases.
5. PEFT significantly reduces cost and training time while maintaining good performance.

Rahul Yadav

9. What is an AI Agent and How Does It Work ?

1 What is an AI Agent ?

- An AI Agent is an autonomous system that can understand a goal, plan steps, use tools, take actions and learn from the results to complete tasks.
- It uses LLMs as the brain but can also interact with external tools, APIs, databases and real-world environments.
- Unlike a simple chatbot, an AI Agent can perform multi-step tasks with reasoning, planning and memory.

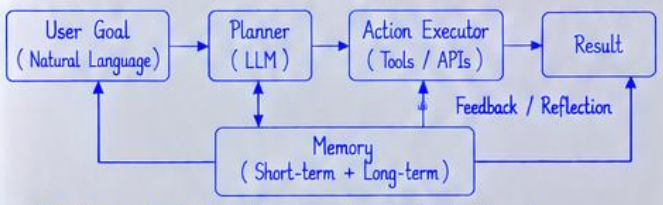
2 Key Characteristics

1. **Goal Oriented** : Works towards a specific goal.
2. **Autonomy** : Can make decisions and take actions.
3. **Tool Use** : Can use external tools and APIs.
4. **Memory** : Remembers past interactions and context.
5. **Planning** : Breaks complex tasks into smaller steps.
6. **Learning** : Improves based on feedback and experience.

3 AI Agent vs Chatbot vs Traditional AI

Aspect	Traditional AI	Chatbot (LLM)	AI Agent
Goal	Specific task	Answer questions	Complete complex tasks
Autonomy	No	Limited	Yes
Tool Use	No	Limited	Yes (APIs, DB, etc.)
Memory	No	Short-term	Short + Long-term
Planning	No	No	Yes
Examples	Rule-based systems	ChatGPT	AutoGPT, LangChain Agents, CrewAI

4 How Does an AI Agent Work ?



1. **Understand Goal** : User provides a high-level goal.
2. **Plan** : Agent breaks the goal into smaller steps.
3. **Take Action** : Uses tools (APIs, search, databases, etc.).
4. **Observe Result** : Checks the output and handles errors.
5. **Reflect and Iterate** : Improves the plan if needed.
6. **Use Memory** : Stores important information for future use.
7. **Final Output** : Provides the final response or completes the task.

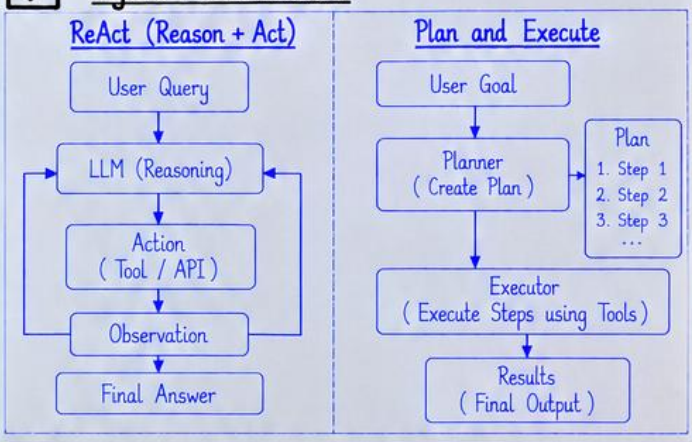
5 Components of an AI Agent

Component	Description	Examples
LLM (Brain)	Understands, reasons and plans	GPT-4, Llama 3, Claude
Planner	Breaks goal into steps	ReAct, Plan-and-Execute
Tools	Interacts with external systems	Web Search, APIs, DB, Code
Memory	Stores past interactions	Conversation Buffer, Vector DB
Executor	Executes actions and handles errors	LangChain AgentExecutor
Feedback Loop	Learns and improves	Self-reflection, Human Feedback

6 Types of AI Agents

1. **Simple Reflex Agent** : Takes action based on current input (no memory).
2. **Model-Based Agent** : Maintains internal state of the environment.
3. **Goal-Based Agent** : Chooses actions to achieve a goal.
4. **Utility-Based Agent** : Optimizes for the best outcome.
5. **Learning Agent** : Learns from experience and feedback.
6. **Multi-Agent System** : Multiple agents work together to solve a complex task.

7 Agent Architectures



8 Common Tools Used by AI Agents

Tool Type	Examples	Use Case
Web Search	SerpAPI, Tavily	Real-time information
Vector Database	FAISS, Pinecone, Qdrant	Long-term memory, RAG
APIs	OpenAI, Slack, Jira, Gmail	Perform actions (send mail, create ticket)
Code Execution	Python REPL, Jupyter	Run code, data analysis
Automation Tools	Zapier, Make	Automate workflows
Frameworks	LangChain, LlamaIndex, CrewAI, AutoGen	Build agent applications

9 Real-world Examples

Application	How AI Agent Helps
Customer Support	Automatically resolve customer queries using tools
Research Assistant	Search papers, summarize and create reports
HR Assistant	Read CVs, schedule interviews, send emails
Developer Assistant	Write code, run tests, fix errors using tools
Data Analysis	Query databases, generate charts and insights
Personal Assistant	Manage calendar, emails and reminders
Multi-Agent System	Multiple agents work together for complex workflows

10 Advantages and Limitations

Advantages	Limitations
• Can perform complex multi-step tasks. • Uses tools to interact with real world. • Saves time and increases productivity. • Can learn and improve over time. • Highly customizable for different use cases.	• May make incorrect decisions. • Depends on tool quality and data. • Can be expensive for complex tasks. • Needs proper monitoring and guardrails. • May face security and privacy risks.

11 Key Takeaways

1. AI Agent = LLM + Planning + Tools + Memory + Feedback.
2. Choose the right architecture (ReAct, Plan-and-Execute, etc.) based on use case.
3. Use appropriate tools and memory for better performance.
4. Monitor, evaluate and add safety guardrails.
5. AI agents are powerful for real-world automation and decision making.

Rahul Yadav

10. How Do You Evaluate the Quality of a Generated Response ?

1 Why Evaluation is Important ?

- Helps ensure responses are accurate, relevant and safe.
- Identifies issues like hallucinations, bias and toxicity.
- Useful for comparing different models and techniques.
- Important for real-world applications (chatbots, RAG systems, AI agents, etc.).
- Helps improve prompts, data and model performance.

2 Key Evaluation Dimensions

1. **Relevance** : How well the response answers the question.
2. **Accuracy** : Factual correctness of the information.
3. **Completeness** : Covers all important points.
4. **Coherence** : Logical flow and readability.
5. **Helpfulness** : Useful and actionable response.
6. **Safety** : No harmful, toxic or biased content.
7. **Hallucination** : Checks if the response contains false information.
8. **Groundedness** : For RAG, checks if the response is based on retrieved context.
9. **Consistency** : Same input should give similar output.
10. **Human Preference** : Overall usefulness and satisfaction.

3 Types of Evaluation Methods

Method	Description	Use Case
Automatic Evaluation	Uses metrics and models	Quick, large scale testing
Human Evaluation	Evaluated by human annotators	High quality, subjective tasks
Model-based Evaluation	Uses LLM as a judge	Evaluate relevance, safety, etc.
Task-specific Evaluation	Custom metrics for specific tasks	Summarization, QA, code generation
A/B Testing	Compare two model outputs	Production systems

4 Common Evaluation Metrics (NLP)

Metric	Formula / Idea	Use Case
BLEU	n-gram overlap with reference	Machine translation
ROUGE	Recall based n-gram overlap	Summarization
METEOR	Considers synonyms	Translation, summarization
BERTScore	Semantic similarity using embeddings	General text generation
Perplexity	Measure of how well model predicts text	Language modeling
Exact Match (EM)	Exact string match	QA (factual)
F1 Score	Harmonic mean of precision & recall	Classification / QA

5 LLM-as-a-Judge Evaluation

- Use a strong LLM (e.g., GPT-4) to evaluate the response.
- Provide clear evaluation rubric and criteria.
- Can evaluate relevance, correctness, helpfulness, tone, etc.
- Useful when human evaluation is expensive or slow.
- Example prompt is shown in section 9.

6 Evaluation Pipeline (Typical Flow)



7 RAG Specific Evaluation

- Check if the response is grounded in retrieved documents.
- Evaluate retrieval quality (recall, precision, hit rate).
- Measure faithfulness (no external hallucination).
- Ensure the response uses correct and relevant context.

Metric	Description	Use Case
Context Precision	Relevant chunks in top-k	Retrieval quality
Context Recall	All relevant chunks retrieved	Retrieval quality
Faithfulness	Response factual to context	RAG response eval
Answer Relevance	Response answers the question	RAG response eval
Hallucination Rate	Measures false information	Safety and quality

8 Example : LLM-as-a-Judge Prompt

```
You are an expert evaluator. Evaluate the following response
based on relevance, accuracy, helpfulness, and safety.
Give a score from 1 to 5 for each criterion and a brief reason.
Question: {user_question}
Response: {model_response}
Evaluation Format:
Relevance: <score> - <reason>
Accuracy: <score> - <reason>
Helpfulness: <score> - <reason>
Safety: <score> - <reason>
Overall Score: <score>
```

9 Example Evaluation (Sample)

Criteria	Score (1-5)	Reason
Relevance	5	Directly answers the question
Accuracy	4	Mostly correct, minor missing details
Helpfulness	5	Clear and actionable
Safety	5	No harmful or biased content
Overall	4.75	High quality response

10 Best Practices

1. Use multiple evaluation methods (automatic + human).
2. Define clear evaluation criteria and rubric.
3. Use task-specific metrics.
4. Check for hallucinations and factual correctness.
5. Test with real-world and edge cases.
6. Continuously monitor and evaluate in production.
7. User feedback is also important.

11 Key Takeaways

- Evaluation ensures accurate, safe and useful responses.
- Use a combination of metrics, LLM judges and human feedback.
- RAG requires retrieval and response evaluation.
- Choose metrics based on the use case and application.
- Continuous evaluation helps in improving model performance.

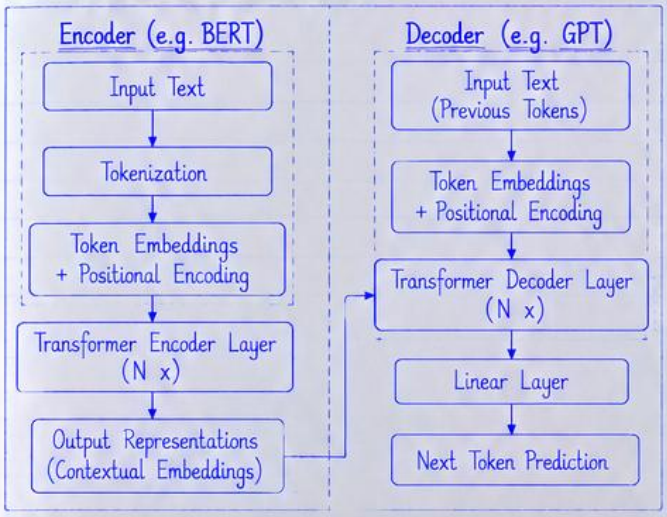
Rahul Yadav

11. Transformer Architecture and Attention Mechanism

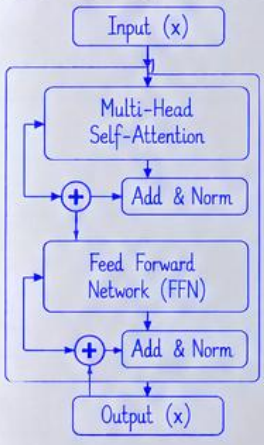
1 What is a Transformer ?

- Transformer is a deep learning architecture introduced in the paper "Attention Is All You Need" (Vaswani et al., 2017).
- It uses self-attention mechanism instead of recurrent or convolutional layers.
- It can process the entire input sequence in parallel.
- It is the foundation of modern LLMs like GPT, BERT, Llama, etc.

2 Overall Architecture



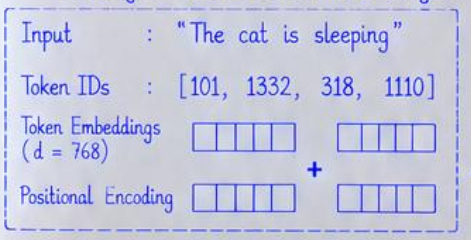
3 Transformer Encoder Layer (Components)



- **Multi-Head Self-Attention** : Helps the model look at different parts of the input sequence simultaneously.
- **Add & Norm** : Residual connection (adding input) + Layer Normalization.
- **Feed Forward Network** : Two linear layers with ReLU activation.
- This block is repeated N times in the transformer.

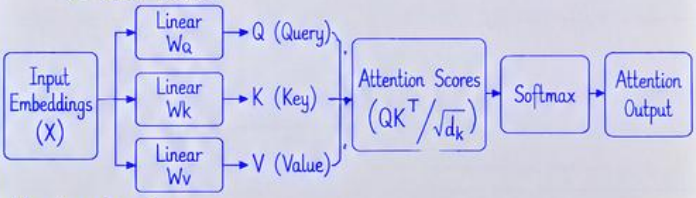
4 Input Embeddings and Positional Encoding

- Input text is converted into tokens using a tokenizer (e.g. BPE, SentencePiece).
- Each token is mapped to a dense vector (embedding).
- Since transformers have no sequence order, positional encoding is added to token embeddings.



5 Self-Attention Mechanism (Single Head)

- Self-Attention helps the model to find the relation between tokens in the input sequence.
- Each token is converted into Query (Q), Key (K) and Value (V) vectors using linear layers.
- Attention score shows how much each token should focus on other tokens.



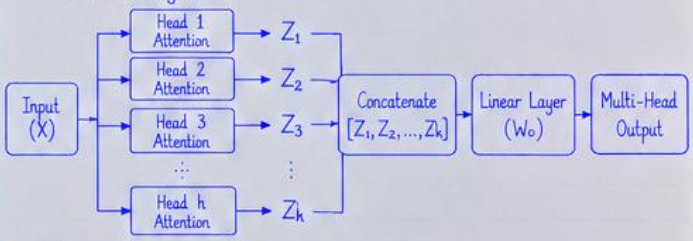
Attention Formula :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Q : What we are looking for
- K : What we are matching against
- V : Actual information to retrieve
- d_k : Dimension of key vectors

6 Multi-Head Attention

- Instead of using a single attention head, multiple heads are used in parallel to capture different types of relationships.
- Outputs from all heads are concatenated and passed through a linear layer.



7 Feed Forward Network (FFN)

- A simple 2-layer fully connected network applied to each token independently and identically.
- Uses a non-linear activation function (usually ReLU or GELU).

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)\ W_2 + b_2$$

- W_1 : Expands the dimension (e.g. 768 → 3072)
- W_2 : Projects back to original dimension (e.g. 3072 → 768)

8 Types of Attention

Type	Description	Use Case
Self-Attention	Each token attends to all tokens in the same sequence	Used in both encoder and decoder
Causal Attention	Token can only attend to previous tokens	Used in decoder (e.g. GPT)
Cross-Attention	Attends to tokens from another sequence (encoder output)	Used in encoder-decoder models (e.g. T5, BART)
Masked Attention	Masks future tokens to prevent information leakage	Used in language modeling (e.g. GPT)

9 Key Takeaways

1. Transformer uses self-attention to understand context.
2. It can process sequences in parallel and is more efficient than RNNs.
3. Attention helps the model focus on important tokens.
4. Multi-head attention captures multiple relationships.
5. Transformer architecture is the foundation of all modern LLMs.

Rahul Yadav

12. Prompt Engineering Techniques

1 What is Prompt Engineering ?

- Prompt engineering is the process of designing instructions and providing the right context to guide an LLM towards useful, accurate and relevant outputs.
- It involves choosing the right words, structure, examples and constraints.
- A well-designed prompt can significantly improve the quality of responses without fine-tuning.

2 Anatomy of a Good Prompt

Component	Description
Role	Define the role of the model (e.g. You are an expert ...)
Task	Clearly state what you want the model to do.
Context	Provide relevant background information.
Constraints	Mention limitations (length, tone, style, etc).
Output Format	Specify the format (bullet points, table, JSON).
Examples	Give sample inputs/outputs if needed (few-shot).

3 Prompt Engineering Techniques

1. **Zero-shot Prompting**
   - Give only the task without examples.
   - Useful for simple tasks and when model already knows the task.
2. **One-shot Prompting**
   - Provide a single example.
   - Helps model understand the expected format.
3. **Few-shot Prompting**
   - Provide multiple examples.
   - Useful for complex tasks or specific output format.
4. **Chain-of-Thought (CoT) Prompting**
   - Ask the model to think step by step.
   - Improves reasoning for complex problems (e.g. math, logic, analysis).
5. **Role Prompting**
   - Assign a specific role to the model (e.g. You are a senior software engineer ...).
   - Helps in getting domain-specific and detailed answers.
6. **Structured Output Prompting**
   - Ask for output in a specific format like JSON, table, etc.
   - Useful for programmatic use and integration with APIs.
7. **Decomposition / Step-back Prompting**
   - Break complex tasks into smaller steps.
   - Helps in solving multi-step problems.
8. **Retrieval-Grounded Prompting (RAG)**
   - Provide relevant retrieved documents as context.
   - Useful for domain-specific and up-to-date information.

4 Example : Before vs After Prompt

Before (Vague Prompt)

Prompt: Explain microservices.

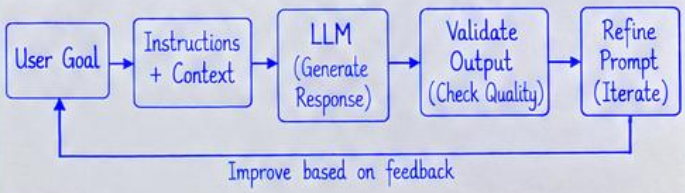
Output: Microservices are an architectural style where applications are built as small, independent services that communicate over a network ...

After (Better Prompt)

Prompt: You are a backend expert. Explain microservices in simple words. Include key characteristics, advantages, disadvantages and a real-world example in bullet points. Keep the answer within 150 words.

Output:
- Definition : ...
- Key Characteristics : ...
- Advantages : ...
- Disadvantages : ...
- Example : ...

5 Prompt Engineering Workflow



6 Common Mistakes

1. Vague or ambiguous requests.
2. Missing important context or constraints.
3. Providing too much irrelevant information.
4. Not specifying the desired output format.
5. Expecting 100% correct answers without verification.
6. Not iterating on the prompt when output is not good.

7 Interview Tips

1. Explain different prompting techniques with examples.
2. Discuss when to use zero-shot, few-shot and CoT.
3. Mention how to design effective prompts for specific use cases.
4. Talk about structured outputs (JSON) and programmatic use.
5. Explain the role of RAG in prompt engineering.
6. Discuss evaluation methods to check response quality.

8 Key Takeaways

1. A clear prompt with proper context leads to better results.
2. Use techniques like few-shot, CoT, role prompting and structured output based on the task.
3. Iterate and refine prompts to improve quality.
4. Combine prompting with RAG for domain-specific knowledge.
5. Prompt engineering is a crucial skill for real-world GenAI applications.

Rahul Yadav

13. Information Retrieval and Vector Databases

13.1 What is Information Retrieval (IR)?

1. Information Retrieval is the process of finding relevant information from a large collection of documents based on a user query.
2. It involves indexing, searching and ranking documents.
3. It is widely used in search engines, RAG systems, chatbots and enterprise knowledge bases.
4. Goal is to retrieve the most relevant documents. that can be used to generate accurate responses.

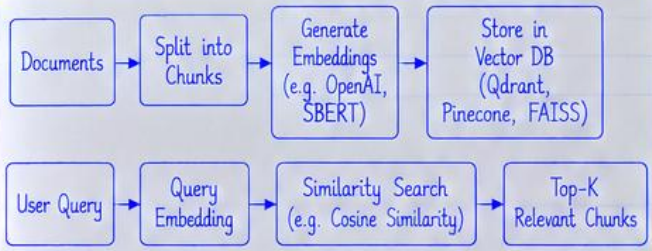
13.2 Traditional IR vs Vector Search

Aspect	Traditional IR (Keyword-based)	Vector Search (Semantic Search)
Representation	Keywords (TF-IDF, BM25)	Dense vectors (Embeddings)
Search Type	Lexical (exact word match)	Semantic (meaning based)
Captures Context	No	Yes
Example	Elasticsearch, Solr	Pinecone, Qdrant, FAISS, Weaviate, Chroma
Best for	Exact keyword search	Semantic understanding, RAG, similar content

13.3 IR Pipeline (Traditional Search)



13.4 Vector Search Pipeline (Semantic Search)



13.5 Document Chunking

1. Large documents are split into smaller chunks before creating embeddings.
2. Common chunk size : 200 - 1000 tokens.
3. Use overlapping chunks (e.g. 50 - 200 tokens overlap).
4. Helps in better context retrieval and reduces information loss.
5. Different strategies : fixed size, semantic chunking, recursive chunking (LangChain).

13.6 Document Embeddings

1. Text is converted into dense numerical vectors using an embedding model (e.g. OpenAI, all-MiniLM-L6-v2).
2. Similar meaning texts have similar vectors (close in vector space).
3. Embeddings capture semantic meaning instead of exact words.
4. Example: "car" and "automobile" will have similar embeddings.

Example Embedding (5 dimensions - simplified)

"Car" : [0.12, -0.34, 0.56, 0.11, -0.22]
"Automobile" : [0.11, -0.32, 0.54, 0.10, -0.20]
"Apple (fruit)" : [-0.45, 0.23, -0.11, 0.67, 0.33]

13.7 Similarity Search and Distance Metrics

1. Common distance metrics:

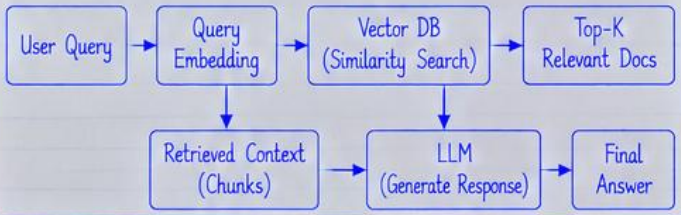
Metric	Formula (Simplified)	Use Case
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$	Most commonly used for text embeddings
Euclidean Distance	$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$	Used in some vector DBs
Dot Product	$A \cdot B$	Used in models like OpenAI embeddings

2. Higher cosine similarity (close to 1) means more similar.
3. We typically retrieve Top-K most similar documents.

13.8 Popular Vector Databases

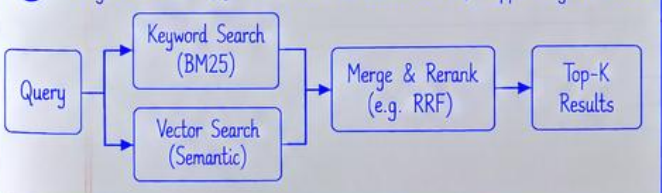
Vector DB	Type	Features	Use Case
Qdrant	Open Source	High performance, filtering, payload	RAG, scalable search
Pinecone	Cloud (Managed)	Easy to use, scalable	Production applications
FAISS	Open Source	Fast similarity search	Local development
Weaviate	Open Source	Hybrid search, GraphQL	Enterprise use cases
Chroma	Open Source	Simple and lightweight	Prototypes and small apps
Milvus	Open Source	Highly scalable	Large scale vector search

13.9 RAG with Vector Database



13.10 Hybrid Search (Keyword + Vector)

1. Combines traditional keyword search (e.g. BM25) with vector search.
2. Helps in better retrieval accuracy.
3. Useful when exact keywords and semantic meaning both matter.
4. Many vector DBs (Qdrant, Weaviate, Elasticsearch) support hybrid search.



13.11 Use Cases

1. RAG systems for chatbots and knowledge assistants.
2. Enterprise document search (policies, manuals, reports).
3. Semantic product search in e-commerce.
4. Code search and developer documentation.
5. Legal and research document retrieval.
6. Multimodal search (text + image embeddings).

13.12 Key Takeaways

1. Vector databases enable semantic search using embeddings.
2. Proper chunking, embedding model and similarity metric are important.
3. Hybrid search can improve retrieval results.
4. Vector search is a key component in RAG systems.
5. Choosing the right vector database depends on scalability, features and use case.

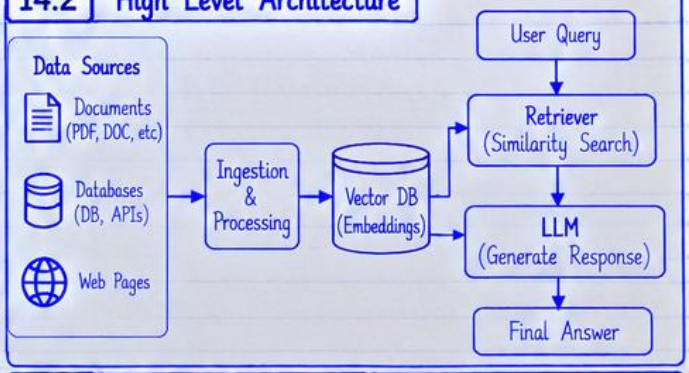
Rahul Yadav

14. Scalable RAG System Design

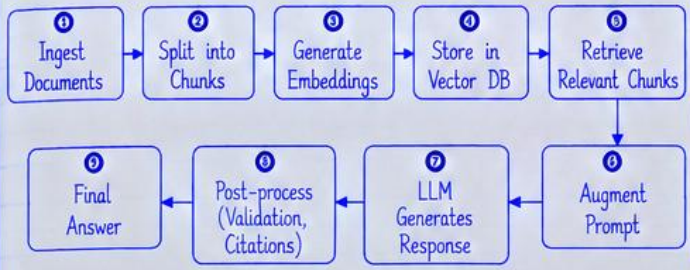
14.1 What is a RAG System ?

1. RAG (Retrieval Augmented Generation) combines information retrieval with LLM generation.
2. It retrieves relevant documents from external sources and provides them as context to the LLM.
3. Helps in generating accurate, up-to-date and grounded responses.
4. Useful for domain-specific knowledge, internal documents and real-time information.

14.2 High Level Architecture



14.3 RAG Workflow (End-to-End Flow)



14.4 Document Ingestion Pipeline



14.5 Chunking Strategies

Strategy	Description	Pros	Cons
Fixed Size	Split by fixed number of tokens	Simple, fast	May break context
Recursive	Split using hierarchy (headings, paragraphs)	Maintains context	Complex implementation
Semantic	Use embedding similarity to split	Better context preservation	Computationally expensive

14.6 Embedding Models

Model	Type	Dimension	Use Case
OpenAI (text-embedding-3)	API (Closed)	1536	General purpose
all-MiniLM-L6-v2	Open Source	384	Lightweight, fast
BGE (BAAI)	Open Source	768	High performance
E5	Open Source	768	Good for RAG
Instructor	Open Source	768	Instruction-based
Cohere Embed	API (Closed)	1024	Multilingual

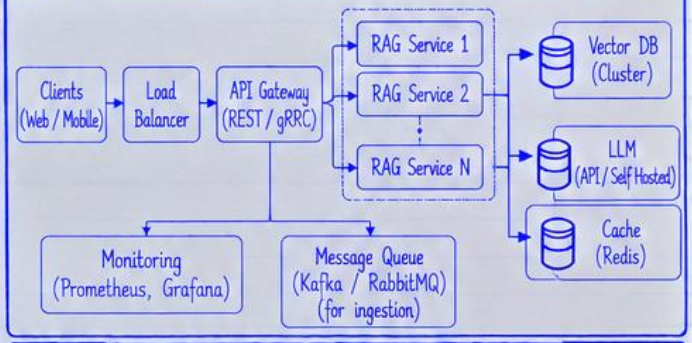
14.9 Similarity Search and Retrieval

Method	Formula / Idea	Use Case
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\lvert A \rvert \lvert B \rvert}$	Most commonly used
Euclidean Distance	$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$	Used in some vector DBs
Dot Product	$A \cdot B$	Used in OpenAI embeddings and dense retrieval
Hybrid Search	Combine BM25 + Vector (Weighted score)	Better results (keyword + semantic)

14.7 Scaling Strategies

1. Use efficient document chunking and embedding models.
2. Use a managed vector database (Pinecone, Qdrant, Weaviate, etc).
3. Implement caching (query cache, embedding cache, response cache).
4. Use asynchronous processing for document ingestion.
5. Use horizontal scaling for API servers (Kubernetes, Docker).
6. Use batch processing for large document ingestion.
7. Optimize retrieval (hybrid search, reranking, metadata filtering).
8. Implement monitoring, logging and rate limiting.

14.8 RAG Deployment Architecture (Scalable)



14.10 RAG Techniques for Better Results

1. Hybrid Search (Keyword + Vector search).
2. Metadata Filtering (filter by source, date, category, etc).
3. Reranking (use cross-encoder like bge-reranker).
4. Query Expansion (rewrite or generate multiple queries).
5. Context Compression (remove irrelevant text).
6. Use citations with source links.
7. Handle long documents with hierarchical retrieval.
8. Use feedback loop to improve retrieval quality.

14.11 Real-world Use Cases

1. Enterprise knowledge search (policies, manuals, internal docs).
2. Customer support chatbots (FAQs, troubleshooting).
3. Code documentation and developer assistants.
4. Legal and compliance document search.
5. Research paper analysis and summarization.
6. Healthcare and medical knowledge systems.
7. E-commerce product search with semantic understanding.

14.12 Key Takeaways

1. RAG combines retrieval and generation for accurate and grounded responses.
2. Choice of chunking, embedding model and vector DB is important.
3. Use hybrid search, reranking and metadata filtering for better results.
4. Scalable architecture requires caching, monitoring and horizontal scaling.
5. RAG is widely used in real-world enterprise applications.

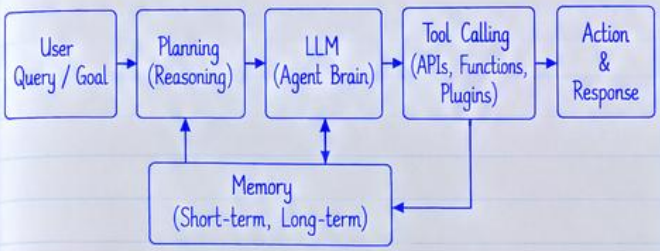
Rahul Yadav

15. AI Agents, Tool Calling and Function Calling

1 What is an AI Agent ?

- An AI agent is a system that can understand a goal, reason, plan steps, use external tools and take actions to achieve the goal.
- It can interact with the environment, use memory and make decisions autonomously.
- LLMs act as the brain and tools/APIs give real-world capabilities.
- Used for automation, research, data analysis, customer support, and complex task execution.

2 AI Agent Architecture



3 Types of AI Agents

Type	Description	Use Case
Simple Reflex Agent	React based on current input	Basic Q&A
Goal-based Agent	Plans steps to achieve a goal	Task automation
Tool-using Agent	Uses external tools/APIs	Real-world data access
Multi-Agent System	Multiple agents work together	Complex workflows
Autonomous Agent	Can plan, execute and learn with minimal input	Research, coding, enterprise automation

4 What is Tool Calling ?

- Tool calling allows the LLM to use external tools like APIs, database, search engines, calculators, or custom functions.
- The LLM decides when to use a tool based on the user query and returns structured input for the tool.
- Tool calling extends the capabilities of LLMs beyond their training data.

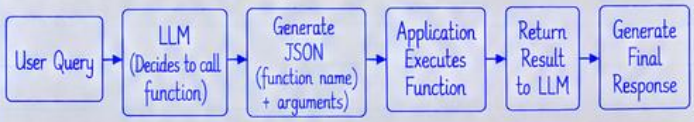
5 What is Function Calling ?

- Function calling is a specific type of tool calling where the LLM generates a structured function call (with defined parameters) instead of natural language.
- It returns a JSON object with function name and arguments.
- Function calling is commonly used with OpenAI, Azure OpenAI and other LLMs for reliable API integration.
- Helps in getting structured, predictable and machine-readable output.

6 Tool Calling vs Function Calling

Aspect	Tool Calling	Function Calling
Definition	General concept to use external tools	Specific implementation using structured function schema
Supported By	LangChain, LlamaIndex, OpenAI, etc.	OpenAI function calling, Azure OpenAI, etc.
Input Format	Can be natural language or structured	Structured JSON with function name and arguments
Output	Tool result (text, JSON, data, etc.).	JSON function call which is executed by the application
Use Case	General tool use (search, API, DB, code, etc.)	Reliable and structured API integration

7 Function Calling Workflow



Example : Weather API Function Call

User Query : What is the weather in Delhi ?

Function Schema :

```
{
{ "name": "get_weather",
  "description": "Get current weather information",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "City name"}
    },
    "required": ["city"]
  }
}
```

LLM Output :

```
{ "name": "get_weather",
  "arguments": {
    "city": "Delhi"
  }
}
```

8 Popular Frameworks and Tools

Tool / Framework	Description	Use Case
LangChain	Tool calling, agents, memory, chains	Build AI agents with tools
LlamaIndex	Data ingestion, RAG, agent framework	Knowledge based agents
OpenAI Function Calling	Structured function calling with JSON	Reliable API integration
CrewAI	Multi-agent framework	Complex agent workflows
AutoGen	Multi-agent conversation framework	Research and automation

9 Key Takeaways

1. AI agents can plan, use tools and take actions to achieve goals.
2. Tool calling allows LLMs to interact with external systems.
3. Function calling provides structured and reliable API integration.
4. Frameworks like LangChain, LlamaIndex and CrewAI help in building agents.
5. AI agents are widely used in real-world applications like automation, research and customer support.

Rahul Yadav

16. MLOps and LLMOps

16.1 What is MLOps ?

- MLOps (Machine Learning Operations) is a set of practices to automate and manage the machine learning model lifecycle in production.
- It helps in building, deploying, monitoring and maintaining ML models at scale.
- Goal is to make ML systems reliable, scalable, reproducible and efficient.
- It bridges the gap between data science and software engineering.

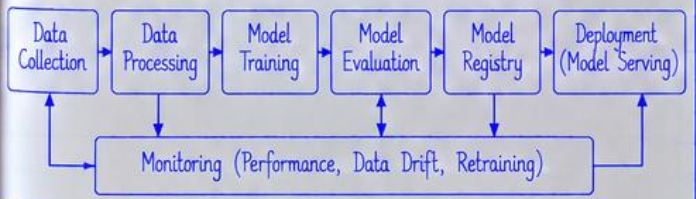
16.2 What is LLMOps ?

- LLMOps is an extension of MLOps for managing Large Language Models (LLMs) and Generative AI applications.
- It involves prompt management, data management, evaluation, deployment, monitoring and cost optimization.
- It helps in building reliable, safe and scalable LLM applications in production.

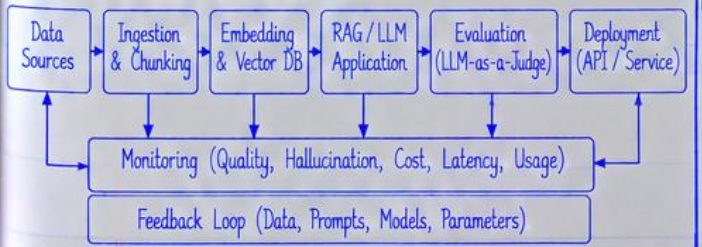
16.3 MLOps vs LLMOps

Aspect	MLOps	LLMOps
Model Type	Traditional ML models	Large Language Models (LLMs)
Data	Structured data	Unstructured data (text, images, etc.)
Key Components	Training pipeline, feature store, model registry	Prompt management, RAG, evaluation, vector DB
Evaluation	Accuracy, Precision, Recall	Human eval, LLM-as-a-judge, safety, groundedness
Deployment	Model serving (REST API)	LLM serving, RAG pipeline
Monitoring	Model performance, data drift	Response quality, hallucination, cost, usage, safety
Use Case	Recommendation, fraud detection, classification	Chatbots, document Q&A, AI agents, content generation

16.4 MLOps Pipeline (Traditional ML)



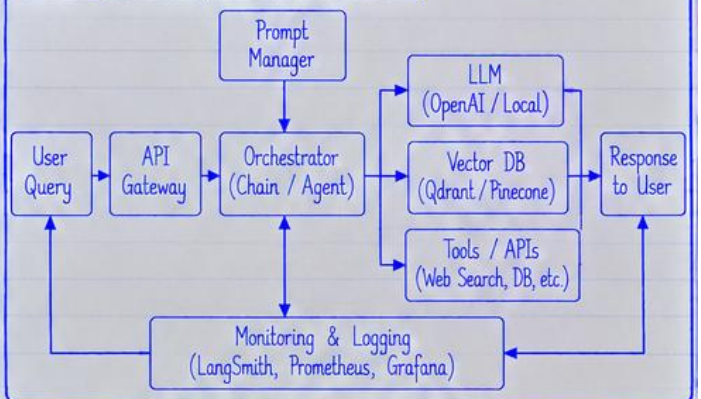
16.5 LLMOps Pipeline



16.6 Key Components of LLMOps

Component	Description	Example Tools
Data Management	Ingestion, cleaning, chunking, versioning	Airflow, LangChain, Unstructured, DVC
Vector Database	Store and search embeddings	Qdrant, Pinecone, Weaviate, FAISS, Chroma
Prompt Management	Version and manage prompts	PromptHub, LangSmith
Model Management	Manage LLM versions and configs	Hugging Face, Ollama, Model Registry
Evaluation	Evaluate response quality	Ragas, LangSmith, LLM-as-a-Judge
Deployment	Serve LLM/RAG applications	FastAPI, Docker, Kubernetes
Monitoring	Track usage, cost, safety, latency	Prometheus, Grafana, LangSmith, Arize
Security	Access control, data privacy	OAuth, API Gateway, Data masking

16.7 LLMOps Architecture



16.8 Monitoring in LLMOps

1. Track response quality (relevance, faithfulness, helpfulness).
2. Monitor hallucinations and unsafe content.
3. Track latency, throughput and error rates.
4. Monitor token usage and cost.
5. Monitor tool usage and API failures.
6. Use user feedback for continuous improvement.

16.9 Challenges

1. Hallucinations and incorrect responses.
2. Evaluation is complex (no single right answer).
3. High cost and latency for large models.
4. Data privacy and security concerns.
5. Scaling embedding and vector databases.
6. Managing prompts, model versions and tools.
7. Ensuring reliability and uptime in production.

16.10 Key Takeaways

1. MLOps and LLMOps help in building production-ready AI systems.
2. LLMOps adds additional components like prompt management, RAG, and evaluation.
3. Monitoring, evaluation and feedback loop are critical.
4. Use the right tools and architecture based on use case.
5. Focus on scalability, security, cost optimization and user experience.

Rahul Yadav

17. Multimodal Generative AI

17.1 What is Multimodal Generative AI ?

1. Multimodal Generative AI can understand, process and generate content across multiple modalities (text, image, audio, video, etc).
2. It can take input from one or more modalities and generate output in the same or different modality.
3. It uses large models trained on multimodal datasets to learn the relationships between different types of data.
4. Examples: GPT-4V, Gemini, Claude 3, LLaVA, DALL·E 3, Stable Diffusion, Whisper, etc.
5. Useful for real-world applications like image understanding, video analysis, document Q&A, and content generation.

17.2 Types of Multimodal Models

Type	Input Modalities	Output Modalities	Example Models
Text-to-Image	Text	Image	DALL·E, Stable Diffusion
Image-to-Text	Image	Text	CLIP, LLaVA, GPT-4V
Text-to-Video	Text	Video	Sora, Runway, Pika
Image-to-Video	Image + Text	Video	Gen-2, Pika, Stable Video
Audio-to-Text	Audio	Text	Whisper
Text-to-Audio	Text	Audio	TTS (XTTS, Bark, etc)
Video Understanding	Video	Text	Gemini, Claude 3, LLaVA

17.3 Common Use Cases

1. Image understanding and description (e.g. OCR, document QA)
2. Visual question answering (VQA)
3. Text-to-image and image-to-video generation
4. Video summarization and analysis
5. Audio transcription and speech generation
6. Multimodal chat assistants (image + text + voice)
7. Content creation (marketing, design, education)
8. Healthcare (medical image analysis)
9. Autonomous systems (robotics, self-driving)
10. E-commerce (image search, product recommendations)

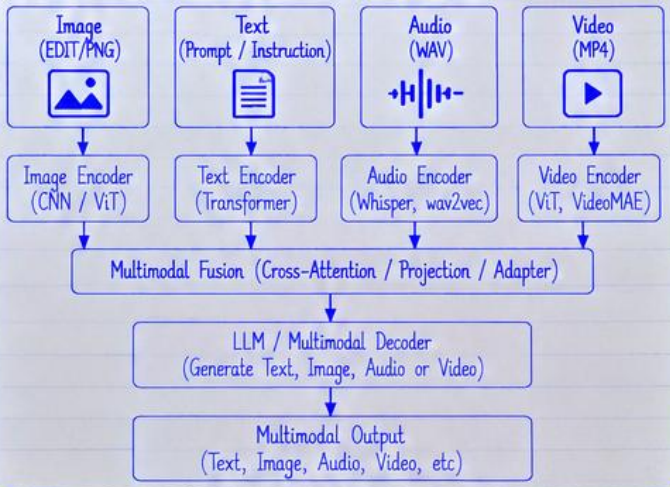
17.4 Multimodal Data and Preprocessing

Modality	Data Format	Preprocessing Techniques	Libraries / Tools
Text	TXT, JSON	Tokenization, cleaning	HuggingFace, tiktoken
Image	JPG, PNG	Resize, normalization	OpenCV, PIL, torchvision
Audio	WAV, MP3	Resampling, feature extraction (Mel spectrogram)	librosa, torchaudio
Video	MP4, AVI	Frame extraction, sampling	OpenCV, decord
Document	PDF, DOC	OCR, parsing, chunking	PyPDF2, Tesseract

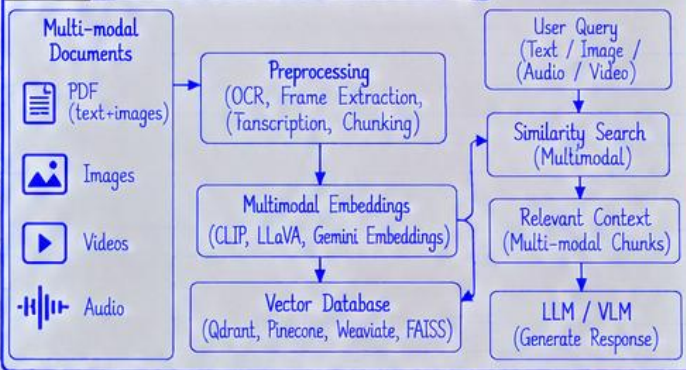
17.6 Popular Multimodal Models and Tools

Model / Tool	Type	Supported Modalities	Use Case
GPT-4V	Closed Source	Text, Image	Visual Q&A, analysis
Gemini	Closed Source	Text, Image, Audio, Video	Multimodal reasoning
Claude 3	Closed Source	Text, Image	Document understanding
LLaVA	Open Source	Text, Image	Visual instruction tuning
DALL·E 3	Closed Source	Text → Image	Image generation
Stable Diffusion	Open Source	Text → Image	Image generation
Whisper	Open Source	Audio → Text	Speech to text
Sora	Closed Source	Text → Video	Video generation
BLIP-2	Open Source	Image + Text	Image captioning, VQA
Qwen-VL	Open Source	Text, Image, Video	Multimodal chat, analysis

17.5 Multimodal Model Architecture



17.7 Multimodal RAG Architecture



17.8 Challenges in Multimodal AI

1. Large computational requirements and high cost.
2. Handling different data modalities and formats.
3. Data alignment and synchronization (e.g. text with video frames).
4. Hallucinations and incorrect understanding of visual/audio content.
5. Limited availability of high-quality multimodal datasets.
6. Evaluation is more complex than text-only models.
7. Privacy and security concerns (images, audio, video may contain sensitive data).
8. Real-time processing and low latency for video/audio.
9. Scalability for production deployment.
10. Ensuring fairness and reducing bias across modalities.

17.9 Evaluation Metrics

Modality	Metrics	Description
Text	BLEU, ROUGE, BERTScore	Text generation quality
Image	CLIP Score, FID, IS	Image generation quality
Video	FVD, CLIP Score	Video generation quality
Audio	WER, MOS	Speech recognition / generation quality
Multimodal	Human Evaluation	Overall multimodal understanding

17.10 Key Takeaways

1. Multimodal Generative AI can work with text, image, audio and video.
2. It enables more natural and intelligent AI applications.
3. Models use specialized encoders and a fusion mechanism to handle multiple modalities.
4. Multimodal RAG helps in searching and understanding different types of content.
5. It has wide real-world applications but also comes with challenges like cost, evaluation and security.
6. Stay updated with latest models, tools and research in this fast-growing field.

Rahul Yadav

18. Security, Privacy, Hallucinations and Responsible AI

18.1 What is AI Security ?

- AI security focuses on protecting AI systems, models, data and infrastructure from threats and misuse.
- It ensures the confidentiality, integrity and availability of AI applications.
- It includes securing prompts, data, model weights, APIs and deployment environments.
- Important for both traditional ML and Generative AI applications.

18.2 What is Privacy in AI ?

- Privacy in AI means protecting user data and sensitive information used for training, fine-tuning or inference.
- It ensures personal and confidential data is not exposed, misused or stored without consent.
- Techniques like data anonymization, differential privacy and federated learning help in preserving privacy.
- Important for compliance with regulations like GDPR, DPDP Act and HIPAA.

18.3 Common Security Threats in Generative AI

Threat	Description
Prompt Injection	Malicious input to manipulate model behavior
Data Leakage	Sensitive data exposed in model output
Model Theft	Stealing model weights or API access
Adversarial Attacks	Crafted inputs to generate incorrect output
Jailbreaking	Bypassing safety restrictions
Insecure Plugins/Tools	Compromised external tools or APIs
Supply Chain Attacks	Attacks on dependencies and libraries

18.4 Privacy Techniques

Technique	Description	Use Case
Data Anonymization	Remove personally identifiable information (PII)	User data, logs
Differential Privacy	Add noise to data to prevent identification	Model training
Federated Learning	Train model on local devices without sharing raw data	Mobile, healthcare
Access Control	Restrict access to data and models	Enterprise systems
Encryption	Encrypt data at rest and in transit	APIs, databases

18.5 What are Hallucinations ?

- Hallucinations are incorrect, misleading or fabricated information generated by an LLM.
- They occur when the model generates output that is not grounded in the given context or real world facts.
- Can happen due to lack of knowledge, ambiguous queries, or model limitations.
- Reducing hallucinations is important for reliable and trustworthy AI applications.

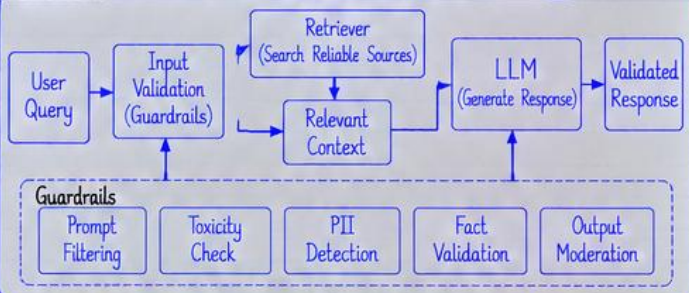
18.6 Types of Hallucinations

Type	Description
Factual Hallucination	Incorrect facts or fake information
Contextual Hallucination	Irrelevant or off-topic response
Citation Hallucination	Generating fake references or sources
Logical Hallucination	Incorrect reasoning or conclusions
Entity Hallucination	Non-existent people, places or objects
Numerical Hallucination	Incorrect numbers, dates or statistics

18.7 Techniques to Reduce Hallucinations

1. Use RAG (Retrieval Augmented Generation) with reliable sources.
2. Provide clear and detailed prompts.
3. Use citation and source attribution.
4. Apply knowledge grounding and context validation.
5. Use model fine-tuning with domain-specific data.
6. Use tools and function calling to fetch real-time information.
7. Implement guardrails and output validation.
8. Combine multiple models or verification steps.

18.8 RAG with Guardrails Architecture



18.9 Responsible AI Principles

1. Fairness – Avoid biased or discriminatory outputs.
2. Accountability – Be responsible for AI decisions.
3. Transparency – Clearly explain how the model works.
4. Privacy – Protect user data and sensitive information.
5. Safety – Prevent harmful or dangerous outputs.
6. Reliability – Ensure consistent and accurate performance.
7. Human-Centric – Keep humans in the loop for critical decisions.
8. Sustainability – Consider environmental impact and cost.

18.10 Legal and Ethical Considerations

Area	Key Points
Data Protection	Follow GDPR, DPDP Act, HIPAA
Bias and Fairness	Avoid biased and discriminatory outputs
Content Safety	Prevent harmful, illegal or toxic content
Copyright and IP	Respect intellectual property rights
Model Usage	Follow acceptable use policies of model providers
Transparency	Inform users when content is AI-generated
Compliance	Follow industry and organizational guidelines

18.11 Evaluation and Monitoring for Safety

Metric	Description
Toxicity Rate	Percentage of harmful or toxic responses
Bias Detection	Measure fairness across different groups
Hallucination Rate	Percentage of incorrect or ungrounded answers
PII Leakage Rate	Percentage of responses containing sensitive data
Content Moderation	Identify and block harmful content
Human Evaluation	Expert review for safety and reliability

18.12 Key Takeaways

1. AI systems must be secure, private and trustworthy.
2. Use techniques like RAG, guardrails and validation to reduce hallucinations.
3. Follow legal, ethical and compliance guidelines.
4. Implement monitoring and evaluation for safety and bias.
5. Responsible AI ensures long-term trust, reliability and adoption.
6. Always consider the impact of AI on users, society and environment.

Rahul Yadav

19. Generative AI Interview Questions and Final Revision

19.1 General Interview Questions (Conceptual)

1. What is Generative AI and how does it work ?
2. What is the difference between LLM and Generative AI ?
3. Explain the Transformer architecture.
4. What are tokens, embeddings and context window ?
6. What is prompt engineering and why is it important ?
6. What is RAG and how does it work ?
7. Explain fine-tuning, LoRA and PEFT.
8. What are AI Agents and how do they work ?
9. What are hallucinations and how to reduce them ?
10. What are the ethical and legal considerations in Generative AI ?

19.2 Technical Interview Questions (Medium)

1. Explain self-attention mechanism with formulas.
2. How does multi-head attention work ?
3. What is the role of positional encoding ?
4. Explain the difference between encoder-only, decoder-only and encoder-decoder models.
5. How does vector database work and why is it used in RAG ?
6. Explain the difference between cosine similarity and dot product.
7. How does function calling / tool calling work in LLMs ?
8. Explain how to design a scalable RAG system.
9. How will you handle large documents for RAG ?
10. What are the challenges in deploying LLM applications in production ?

19.3 Advanced Interview Questions (Hard)

1. How will you optimize retrieval quality in a RAG system ?
2. Design a multi-modal RAG architecture (text, image, audio).
3. How will you evaluate and reduce hallucinations ?
4. Explain model quantization and its benefits.
5. How will you implement caching in LLM applications ?
6. How will you handle high traffic and scaling for LLM APIs ?
7. Explain the trade-offs between fine-tuning, RAG and agents.
8. How will you ensure security and prevent prompt injection ?
9. Design a production-ready LLMOps pipeline.
10. How will you monitor and evaluate LLM applications in production ?

19.4 System Design Questions

1. Design a scalable RAG system for company documents.
2. Design an AI chatbot with memory and tool use.
3. Design a multi-modal AI system (text + image + audio).
4. Design an LLM-based customer support system.
5. Design a document processing pipeline (PDF, images, videos).
6. Design an AI agent system for task automation.
7. Design a system for real-time data ingestion and vector search.
8. Design a complete GenAI platform (from data ingestion to deployment).

19.5 Coding and Implementation Questions

1. Implement document loader and text splitter using LangChain.
2. Create embeddings and store in Qdrant / FAISS.
3. Build a simple RAG pipeline in Python.
4. Implement a function calling / tool use example.
5. Create a multi-modal input example (text + image).
6. Implement chat history memory in LangChain.
7. Write a Streamlit chatbot for PDF Q&A.
8. Implement evaluation for RAG (BLEU / ROUGE / RAGAS).

19.6 Important Diagrams to Revise

1. Transformer Architecture (Encoder & Decoder).
2. Self-Attention and Multi-Head Attention.
3. RAG Architecture (Ingestion → Retrieval → Generation).
4. Vector Database Architecture.
5. AI Agent Architecture (Planning, Tools, Memory).
6. LLMOps Pipeline (Training → Deployment → Monitoring).
7. Multi-Modal Model Architecture.
8. Scalable RAG System Design (Load Balancer, Caching, Vector DB).

19.7 Important Formulas to Remember

1. Scaled Dot-Product Attention : $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
2. Cosine Similarity : $\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|}$
3. Euclidean Distance : $d(A, B) = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$
4. Perplexity : $\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log P(w_i)\right)$
5. Cross Entropy Loss : $L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$

19.8 Key Tools and Frameworks

Category	Tools / Frameworks
LLM Frameworks	LangChain, LlamaIndex, OpenAI, Hugging Face
Vector Databases	Qdrant, Pinecone, Weaviate, FAISS, Chroma
Model Training	PyTorch, Transformers, PEFT, LoRA, DeepSpeed
Serving & Deployment	FastAPI, Docker, Kubernetes, vLLM, Ollama
Monitoring & Observability	Prometheus, Grafana, LangSmith, Arize
Evaluation	RAGAS, BLEU, ROUGE, BERTScore, Human Eval
Multi-modality	OpenCV, Whisper, CLIP, LLaVA, Gemini
Others	Streamlit, Gradio, Airflow, MLflow, DVC

19.9 Final Revision Checklist

1. Revise all concepts from Page 1 to 19.
2. Understand key diagrams and architectures.
3. Practice important interview questions.
4. Build at least 2 end-to-end projects (RAG or Agent).
5. Be familiar with latest models, tools and research trends.
6. Understand system design, scalability and cost optimization.
7. Review security, privacy, ethical and legal considerations.
8. Practice coding and implementation questions.
9. Take mock interviews and improve weak areas.
10. Prepare real-world examples from your experience.

19.10 Key Takeaways

1. Generative AI is a rapidly evolving field with wide applications.
2. Strong understanding of fundamentals, architecture and tools is essential.
3. RAG, fine-tuning and agents enable real-world use cases.
4. Focus on hands-on projects and practical implementation.
5. Evaluation, monitoring, security and responsible AI are critical for production.
6. Multi-modal AI and LLMOps are important for scalable systems.
7. Stay updated with latest trends, research and best practices.
8. Be ready to explain concepts with real-world examples and trade-offs.

Rahul Yadav